

Práctica Kubernetes con Minikube

Informe para sustentación y demostración en vivo

Dato	Valor
Estudiante	Natalia Hernández
Fecha	9 de septiembre de 2026
Entorno	Mac Apple Silicon con Ubuntu 22.04 ARM64
Estado	Ejecutada y verificada

Minikube 1.39.0 y Kubernetes 1.37.0 verificados, con Ingress y rolling update ejecutados correctamente.

Propósito y resultado

Esta práctica demuestra el ciclo de administración de una aplicación en Kubernetes: creación del clúster, Deployments, Services, observación, escalado, balanceo, Ingress y actualización gradual. Todo se ejecutó en Ubuntu ARM64 con Minikube y el controlador Docker.

Minikube 1.39.0 y Kubernetes 1.37.0 verificados, con Ingress y rolling update ejecutados correctamente.

Arquitectura

Componente	Elemento	Función
VM	servidorUbuntu 192.168.100.3	Ubuntu 22.04 ARM64
Clúster	Minikube	Control plane de un nodo sobre Docker
Aplicación 1	hello-minikube	Echo server y NodePort 30081
Aplicación 2	hello-node	Versiones v1 y v2, NodePort 30080
Entrada	Ingress NGINX	Rutas /node y /echo para practica.local
Panel	Kubernetes Dashboard	Acceso temporal mediante proxy

Conceptos que debo explicar

- Pod.** Unidad mínima que ejecuta uno o más contenedores.
- Deployment.** Mantiene réplicas y administra actualizaciones de pods.
- Service.** Proporciona una dirección estable y distribuye tráfico entre pods.
- NodePort.** Publica un Service en un puerto del nodo.
- Ingress.** Aplica reglas HTTP para dirigir rutas o dominios a distintos Services.
- Rolling update.** Reemplaza pods gradualmente y conserva instancias disponibles.

Desarrollo y resultados

Herramientas. Se instalaron binarios ARM64 de Minikube y kubectl. El nodo alcanzó estado Ready.

hello-minikube. El Deployment y su NodePort respondieron con el nombre del pod que atendió la solicitud.

Dashboard. El complemento quedó Ready y un proxy temporal respondió HTTP 200.

hello-node. Se construyeron v1 y v2 dentro del runtime de Minikube. Se inspeccionaron código, registros y variables dentro del pod.

Escalado. El Deployment pasó de una a cuatro réplicas y luego a dos. Doce peticiones mostraron pods distintos.

Ingress. practica.local dirigió /node a hello-node y /echo a hello-minikube.

Actualización. Kubernetes sustituyó gradualmente v1 por v2; las respuestas posteriores confirmaron v2.

Resultados comprobados

Prueba	Resultado	Acceso o detalle
Nodo	Ready	Kubernetes 1.37.0 ARM64
hello-minikube	HTTP correcto	NodePort 30081
Dashboard	HTTP 200	Proxy en el puerto 8001
Port-forward	Respuesta v1	Puerto 8090
Escalado	4 pods	Peticiones repartidas
Ingress	2 rutas correctas	/node y /echo
Rolling update	v1 a v2	Pods reemplazados gradualmente

Guion de demostración en vivo

La demostración debe mostrar primero el estado, después la funcionalidad y finalmente la evidencia. En cada paso explique qué recurso observa y por qué el resultado confirma el objetivo.

Paso	Qué mostrar	Acción	Qué explicar
1	Liberar memoria	Detener Jupyter y servicios no necesarios	Minikube requiere recursos reservados.
2	Iniciar el clúster	minikube start --driver=docker	Espere a que control plane y kubelet estén Running.
3	Mostrar el nodo	kubectl get nodes -o wide	Explique Ready, versión y runtime containerd.
4	Aplicar manifiestos	kubectl apply -f hello-minikube y hello-node	Deployment crea pods; Service publica el acceso.
5	Escalar	kubectl scale deployment/hello-node --replicas=4	kubectl get pods muestra cuatro réplicas.
6	Probar balanceo	Varias peticiones al Service	Las respuestas incluyen nombres de pod diferentes.
7	Mostrar Ingress	curl con Host practica.local	Las rutas /node y /echo llegan a servicios distintos.
8	Actualizar	kubectl set image a hello-node:v2	rollout status y las respuestas confirman v2.

Evidencias para abrir si la red falla

- evidencias/01-versiones-cluster.txt: versiones y control plane.
- evidencias/20-balanceo-cuatro-pods.txt: doce respuestas de pods distintos.
- evidencias/22-ingress-rutas.txt: las dos rutas del Ingress.
- evidencias/27-rolling-update-v2-http.txt: respuestas de la versión v2.
- evidencias/30-minikube-estado-final.txt: clúster detenido de forma segura.

Preguntas probables y respuestas

¿Qué diferencia hay entre Pod y Deployment?

El pod ejecuta contenedores; el Deployment crea y reemplaza pods para mantener el estado deseado.

¿Por qué se necesita un Service?

Los pods cambian de IP al recrearse. El Service conserva una dirección estable y balancea las solicitudes.

¿Qué hace Ingress?

Recibe HTTP y aplica reglas de host o ruta para enviar cada petición al Service correcto.

¿Cómo funciona el rolling update?

El Deployment crea nuevos pods y retira los anteriores por etapas según maxSurge y maxUnavailable.

¿Por qué no se ejecutó el ZIP de GPU?

El ejemplo necesita una GPU NVIDIA visible en Linux y sus controladores. VirtualBox sobre Apple Silicon no expone ese dispositivo.

Idea de cierre

La práctica no se limita a mostrar contenedores activos. La evidencia demuestra que la configuración es reproducible, que la aplicación responde desde otra máquina y que los datos o servicios se mantienen según el mecanismo estudiado.

Lista de preparación

- ☐ Conectar el Mac a la corriente y cerrar aplicaciones pesadas.
- ☐ Confirmar que servidorUbuntu y clienteUbuntu aparecen como running.
- ☐ Abrir este informe, el README del proyecto y una terminal.
- ☐ Ejecutar la verificación antes de que comience la sustentación.
- ☐ Mantener abiertas solamente las páginas de esta práctica.
- ☐ Conservar la carpeta evidencias como respaldo si falla la red o una descarga.

Riesgos y recuperación

- Detenga Jupyter, Flask y bases de datos antes de iniciar Minikube para evitar falta de memoria.
- La IP interna de Minikube se prueba desde servidorUbuntu. Para el navegador del Mac use kubectI port-forward en 8090 y el proxy del Dashboard en 8001.
- Al terminar elimine los recursos y ejecute minikube stop; el perfil se conserva para otra demostración.

Referencias

- Documento 2026-01 Practica Kubernetes-minikube.doc.
- <https://minikube.sigs.k8s.io/docs/start/>
- <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- <https://kubernetes.io/docs/concepts/services-networking/ingress/>